

Linux Essentials

Phase 1 — Filesystem Fundamentals

2. Understand Files and Directories

First understand that Linux organizes almost everything underneath one root:

```
/
```

Unlike Windows:

```
C:\  
D:\  
E:\
```

Linux looks roughly like:

```
/  
├─ home  
├─ etc  
├─ var  
├─ usr  
├─ opt  
├─ tmp  
├─ dev  
├─ proc  
└─ root
```

Before memorizing directories, learn:

```
Absolute path  
Relative path  
Current working directory  
Parent directory  
Home directory  
Root directory
```

Example:

```
/home/aditya/projects/app.jar
```

is an absolute path.

Whereas:

```
projects/app.jar
```

is relative to your current directory.

Essential commands

Only learn these initially:

```
pwdlschmodkdirotouchcpmvrmcat  
less  
head  
tail
```

Also learn:

```
ls-lls-a
```

You don't need dozens of file commands yet.

Phase 2 — Linux Filesystem Hierarchy

3. Learn Only Important Linux Directories

You don't need to memorize the entire Filesystem Hierarchy Standard.

Focus on these.

```
/home
```

Normal users' home directories.

```
/home/aditya  
/home/rohit
```

Comparable conceptually to user folders on Windows.

/root

Home directory of the **root** user.

```
/root
```

Don't confuse:

```
/
```

with:

```
/root
```

They are different.

/etc

System and application configuration.

Very important for DevOps.

Examples:

```
/etc/nginx/  
/etc/ssh/  
/etc/systemd/  
/etc/hosts
```

Mental model:

```
/etc  
≈ configuration
```

/var

Data that changes while the system runs.

Especially important:

```
/var/log
```

For logs.

You may encounter:

```
/var/log/nginx/  
/var/lib/docker/  
/var/lib/mysql/
```

Mental model:

```
/var  
≈ runtime / changing application data
```

/tmp

Temporary files.

```
/tmp
```

/opt

Optional/third-party applications.

For example:

```
/opt/myapp
```

This is often a nice location for manually deployed software.

/usr

Installed programs, libraries and shared resources.

You'll frequently encounter:

```
/usr/bin  
/usr/local/bin
```

For DevOps purposes, understand what it represents rather than memorizing every subdirectory.

/proc

This one is conceptually important.

It is not a normal disk directory.

Linux exposes information about the running kernel and processes through it.

Example:

```
/proc/1234
```

might represent process ID **1234**.

This helps build the connection:

```
Linux exposes many system concepts as files.
```

/dev

Represents devices.

Examples:

```
/dev/null  
/dev/sda
```

Again, understand the idea rather than diving deeply into device management.

Phase 3 — Finding and Inspecting Things

Before permissions, learn how DevOps engineers actually navigate servers.

4. Finding Files

Learn:

```
find
```

Example:

```
find /var/log-name "*.log"
```

Also understand:

```
which java
```

Result could be:

```
/usr/bin/java
```

This answers:

| Which executable will Linux run when I type `java`?

5. Viewing Files and Logs

Very important:

```
cat  
less  
head  
tail
```

Especially:

```
tail-f application.log
```

because DevOps work frequently means:

```
deploy application  
  ↓  
application fails  
  ↓  
open logs  
  ↓  
watch logs live
```

Phase 4 — Users and Groups

Do this **before permissions**.

Otherwise `chmod` and `chown` become meaningless commands students memorize.

6. Understand Linux Users

Linux is a multi-user operating system.

Imagine:

```
Server  
|  
├─ root  
└─ ubuntu
```

```
|— deploy
|— nginx
└─ mysql
```

Different processes can run as different users.

Example:

```
nginx process → nginx user
mysql process → mysql user
your shell    → ubuntu user
```

Learn:

```
whoami
id
```

7. Understand Groups

Groups let multiple users share access.

Example:

```
developers
|— aditya
|— rohit
└─ harshita
```

Then files can belong to:

```
Owner: aditya
Group: developers
```

This prepares you for Linux permissions.

You only need basic familiarity with:


```
id
groups
```

User creation commands can come later.

Phase 5 — Ownership

Now explain that every file has ownership.

Run:

```
ls-l
```

You may see:

```
-rw-r--r-- 1 aditya developers 2048 Aug 8 app.conf
```

The important part:

```
aditya developers
  ↓      ↓
owner  group
```

Every file essentially has:

```
File
├─ Owner
├─ Group
└─ Permissions
```

Phase 6 — Linux Permissions

Now `chmod` finally makes sense.

8. Read, Write and Execute

Three basic permissions:

```
r = read  
w = write  
x = execute
```

And Linux defines them for:

```
user  
group  
others
```

So:

```
rwXr-xr--
```

means:

```
User    → rwX  
Group   → r-x  
Others  → r--
```

Spend significant time understanding this.

It is far more important than memorizing syntax.

9. Directory Permissions

This is frequently taught badly.

For files:

```
r → read file  
w → modify file  
x → execute file
```

For directories the meaning changes slightly.

Conceptually:

```
r → list directory contents  
w → create/delete entries  
x → enter/traverse directory
```

DevOps engineers should understand this distinction.

Phase 7 — chmod

Only now introduce:

```
chmod
```

Teach both approaches but emphasize numeric permissions because you'll encounter them everywhere.

```
r = 4  
w = 2  
x = 1
```

Therefore:

```
7 = rwx  
6 = rw-  
5 = r-x  
4 = r--
```

Example:

```
chmod755 deploy.sh
```

means:

```
Owner   → 7 → rwx
Group   → 5 → r-x
Others  → 5 → r-x
```

And:

```
chmod600 secret.key
```

means:

```
Owner   → rw-
Everyone else → ---
```

Useful DevOps examples:

```
chmod+x deploy.shchmod755 script.shchmod600 private-key.pem
```

Don't spend much time on exotic permission combinations.

Phase 8 — chown and chgrp

After permissions, ask:

| What if permissions are correct, but the wrong user owns the file?

That's where:

```
chown
```

comes in.

Example:

```
sudo chown ubuntu:developers app.jar
```

Meaning:

```
owner → ubuntu  
group → developers
```

Teach:

```
chown  
chgrp
```

But `chown user:group` is usually enough for practical DevOps usage.

Phase 9 — sudo and root

Now introduce privileged operations.

Linux separates:

```
normal user  
  ↓  
limited permissions  
  
root  
  ↓  
almost unrestricted access
```

Instead of permanently working as root:

```
sudo command
```

Example:

```
sudo systemctlrestart nginx
```

Students should understand **why sudo is needed**, rather than treating `sudo` as something added whenever a command fails.

Phase 10 — Processes

Now switch from:

```
Things stored on disk
```

to:

```
Things currently executing
```

This distinction is essential.

For example:

```
/usr/bin/java
```

is a program stored on disk.

When executed:

```
java-jar app.jar
```

Linux creates a:

```
Process
```

with a unique:

PID

Teach:

Program → static executable/file
Process → running instance of program

Phase 11 — Process Inspection

Essential commands:

```
psps aux
```

Optionally:

```
htop
```

if available.

Important things students should recognize:

```
PID  
USER  
CPU  
MEM  
COMMAND
```

Don't teach all `ps` flags.

Example:

```
ps aux |grep java
```

This is far more useful in practice.

Phase 12 — Foreground and Background Processes

Run:

```
java-jar app.jar
```

Your terminal is occupied.

That's a:

```
foreground process
```

Then:

```
java-jar app.jar &
```

runs it in the background.

Teach:

```
jobs  
fg  
bg  
&
```

Only at a basic level.

The important question becomes:

| What happens if I close my SSH session?

This leads naturally to long-running services.

Phase 13 — Process Signals

Instead of thinking:

```
kill = destroy process
```

teach:

```
kill sends a signal to a process.
```

Example:

```
kill1234
```

typically asks the process to terminate gracefully.

And:

```
kill-91234
```

forces it.

Teach primarily:

```
SIGTERM  
SIGKILL
```

And the practical difference:

```
SIGTERM  
↓  
Please terminate gracefully  
  
SIGKILL  
↓  
Kernel immediately stops process
```

DevOps engineers should avoid instinctively using `kill -9`.

Phase 14 — Why Background Processes Aren't Enough

Now create the real problem.

Imagine running:

```
java-jar app.jar &
```

Problems:

What happens after reboot?

What if the application crashes?

Who restarts it?

Where do logs go?

How do we consistently start/stop it?

This naturally introduces:

Services

A service is essentially a program intended to run continuously or in the background.

Examples:

```
nginx  
mysql  
docker  
ssh  
jenkins
```

Many are also called:

```
daemons
```

You don't need a deep historical discussion of daemons.

Phase 15 — systemd

Now students finally understand **why systemd exists**.

Conceptually:

```
Linux boots
  ↓
systemd starts
  ↓
systemd starts required services
```

systemd can:

```
start services
stop services
restart services
automatically start them at boot
track their state
manage dependencies
collect logs
restart failed services
```

Phase 16 — systemctl

This is the part DevOps engineers use constantly.

Learn:

```
systemctl status nginx
systemctlstart nginx
systemctlstop nginx
```

```
systemctl restart nginx
systemctl enable nginx
systemctl disable nginx
```

Understand the critical difference:

```
start
↓
Start it right now

enable
↓
Configure it to start automatically on boot
```

Therefore:

```
sudo systemctl enable nginx
sudo systemctl start nginx
```

do different things.

Phase 17 — systemd Unit Files

Learn the concept, but don't dive into every directive.

A service may have:

```
/etc/systemd/system/myapp.service
```

Example:

```
[Unit]
Description=My Spring Boot Application

[Service]
User=ubuntu
ExecStart=/usr/bin/java -jar /opt/myapp/app.jar
Restart=always
```

```
[Install]
WantedBy=multi-user.target
```

Students should understand:

```
User
↓
Which Linux user runs the process?

ExecStart
↓
What command starts it?

Restart
↓
Should systemd restart it after failure?

WantedBy
↓
When should this service participate in startup?
```

You do **not** need to teach every `systemd` option.

Phase 18 — Logs with journalctl

Once a service fails, students will ask:

| Why did it fail?

That's where:

```
journalctl
```

comes in.

Essential commands:

```
journalctl-u nginx
journalctl-u nginx-f
```

For a Spring Boot service:

```
journalctl-u myapp-f
```

This closes the loop:

```
Deploy application
↓
systemd starts application
↓
application becomes process
↓
process runs as Linux user
↓
permissions control resources
↓
journalctl shows logs
```

The Complete DevOps Linux Roadmap

If I were structuring this as your learning sequence, I'd use these **9 modules**:

Module	Topic	Learn
1	Linux Mental Model	Kernel, OS, shell, server, filesystem
2	Filesystem Navigation	<code>/</code> , paths, <code>pwd</code> , <code>ls</code> , <code>cd</code> , file operations
3	Filesystem Hierarchy	<code>/home</code> , <code>/etc</code> , <code>/var</code> , <code>/opt</code> , <code>/usr</code> , <code>/tmp</code> , <code>/proc</code> , <code>/dev</code>
4	Users & Groups	user, group, root, <code>whoami</code> , <code>id</code> , <code>sudo</code>
5	Permissions & Ownership	<code>rx</code> , user/group/others, <code>chmod</code> , <code>chown</code>
6	Processes	program vs process, PID, <code>ps</code> , <code>top</code> , foreground/background
7	Process Control	signals, <code>kill</code> , SIGTERM, SIGKILL
8	Services & systemd	daemon, service, systemctl, unit files, boot

Module	Topic	Learn
9	Troubleshooting	<code>journalctl</code> , <code>tail -f</code> , permissions, service status

What I would deliberately not teach yet

For a DevOps fundamentals course, don't overload yourself with:

```
Linux kernel internals
Kernel modules
Device driver development
Advanced disk partitioning
LVM internals
SELinux deep dive
AppArmor deep dive
PAM internals
Advanced ACLs
Advanced process scheduling
systemd target internals
cgroups internals
namespace internals
compile-your-own-kernel
advanced shell internals
```

Some of these become relevant later, especially **cgroups and namespaces** when you teach **Docker**, but they make much more sense there.

Your target after completing this roadmap should be simple:

Give yourself a fresh Linux server and you should comfortably be able to:

```
SSH into server
↓
navigate filesystem
↓
find configuration/application files
↓
inspect permissions
↓
fix ownership/permissions
↓
identify running processes
↓
stop a problematic process
```

```
↓  
start/restart a service  
↓  
configure service startup  
↓  
inspect service logs  
↓  
diagnose why application isn't running
```

If you can do that, your Linux foundation is already **more than sufficient to move into Docker, Jenkins, AWS, Kubernetes and CI/CD**. The next Linux topics should then be learned contextually as those tools require them.